ANNÁVE PDF Engine Reference

Design pattern

The engine uses **hexagonal architecture** (ports and adapters). The domain core — the six-stage conversion pipeline — has no knowledge of HTTP, file I/O, or any specific output format. This means the same pipeline can be driven by an *HTTP server*, a *CLI command*, or a test, without any changes to the core logic.

The `code` field in an error response is the *machine-readable and stable identifier*. Do not match on the message text: it may change between versions, and treating it as stable unstable text is a common client bug.

See the Semantic Versioning policy and the standing `ENGINE_ERR_SLUG` naming convention described in the error codes reference.

[Image: Pipeline diagram]

> *All configuration is embedded at build time. No external files are read at runtime. Edit the YAML source and rebuild — this is intentional, not an oversight.*

Six pipeline stages

1. Normalise — strip BOM, unify line endings, enforce the character limit
2. Parse — select a parser by format hint or magic-byte detection
3. Validate — check AST structure and enforce the node count limit
4. Layout — compute element positions from font metrics and page width
5. Paginate — group layout boxes into pages within the height limit
6. Render — drive gopdf to produce a PDF byte stream

- Normalise: internal/engine/normalizer.go
- Parse: internal/parser/registry.go
- Validate: internal/engine/validator.go

```
go
type DocumentParser interface {
CanParse(input string) bool
Parse(input string) (*ast.DocumentNode, error)
}
```

Error response shape

Code	HTTP	Stage
ENGINEERRFILETOOLARGE	400	input

Code	HTTP	Stage
ENGINEERRPARSE_FAILED	422	parser
ENGINEERRINTERNAL	500	render

Fenced code blocks preserve the language hint in the AST `Lang` field, but the renderer does not apply syntax highlighting yet.